6901012610196 Chonpatchut Boonpalit Cpr.E S.2

1. Graphic Mode ข้อ 1: การวาดรูปและสร้าง Animation

วัตถุประสงค์ของข้อ

ข้อนี้เป็นการทดลองวาดรูปและสร้าง Animation ใน Processing เพื่อสังเกตการเปลี่ยนแปลงของตำแหน่งวัตถุเมื่อโปรแกรมทำงานซ้ำอย่างต่อเนื่อง โดยใช้ตัวแปรเก็บตำแหน่งและความเร็วของวัตถุ

โค้ด

```
x = 100
speed = 3


def setup():
    size(600, 400)


def draw():
    global x, speed

    background(255)

    ellipse(x, 200, 80, 80)

    x = x + speed

    if x >= 560 or x <= 40:
        speed = speed * -1
```

อธิบายแต่ละบรรทัด

`x = 100`
กำหนดตำแหน่งเริ่มต้นในแนวนอนของวงกลมให้เท่ากับ 100

`speed = 3`
กำหนดความเร็วของวงกลมให้เคลื่อนที่ครั้งละ 3 หน่วย

`def setup():`
สร้างฟังก์ชัน `setup()` ซึ่งจะทำงานเพียงครั้งเดียวเมื่อเริ่มโปรแกรม

`size(600, 400)`
กำหนดขนาดหน้าต่างเป็นกว้าง 600 พิกเซล และสูง 400 พิกเซล

`def draw():`
สร้างฟังก์ชัน `draw()` ซึ่ง Processing จะเรียกทำงานซ้ำอย่างต่อเนื่อง ทำให้สามารถสร้าง Animation ได้

`global x, speed`
ประกาศว่าจะใช้ตัวแปร `x` และ `speed` ที่อยู่ภายนอกฟังก์ชัน และสามารถเปลี่ยนค่าของตัวแปรเหล่านั้นได้

`background(255)`
ล้างหน้าจอด้วยสีขาวในแต่ละรอบของการวาด เพื่อไม่ให้เกิดรอยของวงกลม

`ellipse(x, 200, 80, 80)`
วาดวงกลมที่ตำแหน่ง `x` และ `y = 200` โดยมีขนาดกว้างและสูง 80 พิกเซล

`x = x + speed`
เพิ่มตำแหน่งของวงกลมตามค่าความเร็ว ทำให้วงกลมเคลื่อนที่

`if x >= 560 or x <= 40:`
ตรวจสอบว่าวงกลมเคลื่อนที่ใกล้ถึงขอบด้านขวาหรือด้านซ้ายหรือไม่

`speed = speed * -1`
เปลี่ยนเครื่องหมายของความเร็ว เช่น จาก 3 เป็น -3 ทำให้วงกลมเปลี่ยนทิศทางและเคลื่อนที่กลับ

ผลการทำงาน

เมื่อโปรแกรมเริ่มทำงาน วงกลมจะเคลื่อนที่ในแนวนอน และเมื่อถึงขอบหน้าจอจะเปลี่ยนทิศทางกลับไปมาอย่างต่อเนื่อง

Error ที่พบและวิธีแก้ไข

1. วงกลมไม่เคลื่อนที่
สาเหตุอาจเกิดจากลืมเขียนคำสั่ง

x = x + speed

วิธีแก้ไขคือเพิ่มคำสั่งนี้ใน `draw()` เพื่อเปลี่ยนตำแหน่งของวงกลมทุกครั้งที่หน้าจอถูกวาด

2. เกิด Error เกี่ยวกับตัวแปร x
หากเปลี่ยนค่าของ `x` ภายใน `draw()` โดยไม่ใช้ `global x` อาจเกิดปัญหาในการอ้างอิงตัวแปร

วิธีแก้ไขคือเพิ่ม

global x, speed

ภายในฟังก์ชัน `draw()`

---

2. Graphic Mode ข้อ 2: วาดเครื่องหมายบวกหลายรูปและเพิ่ม Animation/Interaction

วัตถุประสงค์ของข้อ

ข้อนี้เป็นการประยุกต์ใช้ฟังก์ชัน `draw_positive()` เพื่อวาดเครื่องหมายบวกหลายรูป โดยใช้ List เก็บตำแหน่งและความเร็วของแต่ละรูป และใช้ `while` ในการวนวาดเครื่องหมายบวกทีละรูป นอกจากนี้ยังเพิ่ม Animation ให้เครื่องหมายบวกเคลื่อนที่ และเพิ่ม Interaction โดยทำให้เครื่องหมายบวกมีขนาดใหญ่ขึ้นเมื่อเมาส์เข้าใกล้

โค้ด

```
square_size = 30
number_of_positive = 5

x_position = [110, 330, 550, 770, 990]
speed = [2, -3, 4, -2, 3]


def setup():
    size(1200, 300)
    rectMode(CENTER)


def draw():
    background(200)

    count = 0

    while count < number_of_positive:

        x = x_position[count]
        y = 150

        distance = dist(mouseX, mouseY, x, y)

        if distance < 100:
            current_size = square_size * 2
        else:
            current_size = square_size
```

```
        draw_positive(x, y, current_size)

        x_position[count] = x_position[count] + speed[count]

        if x_position[count] > width - current_size:
            speed[count] = speed[count] * -1

        if x_position[count] < current_size:
            speed[count] = speed[count] * -1

        count = count + 1


def draw_square(x, y, size):
    fill(200)
    stroke(100)
    rect(x, y, size, size)


def draw_positive(x, y, size):

    draw_square(x, y, size)

    draw_square(x, y - size, size)

    draw_square(x, y + size, size)

    draw_square(x - size, y, size)

    draw_square(x + size, y, size)
```

อธิบายแต่ละบรรทัด

```
square_size = 30
```
กำหนดขนาดเริ่มต้นของสี่เหลี่ยมแต่ละอันให้มีขนาด 30

```
number_of_positive = 5
```
กำหนดจำนวนเครื่องหมายบวกทั้งหมดเป็น 5 รูป

```
x_position = [110, 330, 550, 770, 990]
```
สร้าง List เพื่อเก็บตำแหน่งแกน X ของเครื่องหมายบวกแต่ละรูป

```
speed = [2, -3, 4, -2, 3]
```
สร้าง List เพื่อเก็บความเร็วของเครื่องหมายบวกแต่ละรูป โดยค่าบวกหมายถึงเคลื่อนที่ไปทางขวา และค่าลบหมายถึงเคลื่อนที่ไปทางซ้าย

`def setup():`
สร้างฟังก์ชันสำหรับตั้งค่าเริ่มต้นของโปรแกรม

`size(1200, 300)`
กำหนดหน้าต่างให้มีความกว้าง 1200 และความสูง 300 พิกเซล

`rectMode(CENTER)`
กำหนดให้พิกัด `x, y` ของสี่เหลี่ยมเป็นตำแหน่งกึ่งกลางของสี่เหลี่ยม

`def draw():`
สร้างฟังก์ชันที่ทำงานซ้ำตลอดเวลาเพื่อใช้สร้าง Animation

`background(200)`
กำหนดพื้นหลังเป็นสีเทาและล้างภาพเก่าในแต่ละรอบ

`count = 0`
กำหนดตัวแปร `count` ให้เริ่มต้นที่ 0 เพื่อใช้เป็น index ของ List

`while count < number_of_positive:`
วนซ้ำเพื่อทำงานกับเครื่องหมายบวกทีละรูป จนกว่าจะครบจำนวนที่กำหนด

`x = x_position[count]`
นำตำแหน่ง X ของเครื่องหมายบวกตาม index ปัจจุบันมาเก็บในตัวแปร `x`

`y = 150`
กำหนดตำแหน่ง Y ของเครื่องหมายบวกทั้งหมดให้อยู่ตรงกลางหน้าจอ

`distance = dist(mouseX, mouseY, x, y)`
คำนวณระยะห่างระหว่างตำแหน่งเมาส์กับจุดกึ่งกลางของเครื่องหมายบวก

`if distance < 100:`
ตรวจสอบว่าเมาส์อยู่ใกล้เครื่องหมายบวกในระยะน้อยกว่า 100 พิกเซลหรือไม่

`current_size = square_size * 2`
หากเมาส์อยู่ใกล้ ให้เพิ่มขนาดของเครื่องหมายบวกเป็น 2 เท่า

`else:`
กรณีที่เมาส์ไม่ได้อยู่ใกล้เครื่องหมายบวก

`current_size = square_size`
ใช้ขนาดปกติของเครื่องหมายบวก

`draw_positive(x, y, current_size)`
เรียกฟังก์ชันสำหรับวาดเครื่องหมายบวก

`x_position[count] = x_position[count] + speed[count]`
เปลี่ยนตำแหน่งของเครื่องหมายบวกตามความเร็ว ทำให้เกิดการเคลื่อนที่

`if x_position[count] > width - current_size:`
ตรวจสอบว่าเครื่องหมายบวกเคลื่อนที่ใกล้ถึงขอบด้านขวาหรือไม่

`speed[count] = speed[count] * -1`
หากชนขอบ ให้เปลี่ยนเครื่องหมายของความเร็วเพื่อให้เคลื่อนที่กลับ

`if x_position[count] < current_size:`
ตรวจสอบว่าเครื่องหมายบวกเคลื่อนที่ใกล้ถึงขอบด้านซ้ายหรือไม่

`count = count + 1`
เพิ่มค่า `count` เพื่อไปทำงานกับเครื่องหมายบวกตัวถัดไป

`def draw_square(x, y, size):`
สร้างฟังก์ชันสำหรับวาดสี่เหลี่ยมหนึ่งรูป

`fill(200)`
กำหนดสีภายในของสี่เหลี่ยม

`stroke(100)`
กำหนดสีเส้นขอบของสี่เหลี่ยม

`rect(x, y, size, size)`
วาดสี่เหลี่ยมตามตำแหน่งและขนาดที่กำหนด

`def draw_positive(x, y, size):`
สร้างฟังก์ชันสำหรับวาดเครื่องหมายบวก

`draw_square(x, y, size)`
วาดสี่เหลี่ยมตรงกลาง

`draw_square(x, y - size, size)`
วาดสี่เหลี่ยมด้านบน

`draw_square(x, y + size, size)`
วาดสี่เหลี่ยมด้านล่าง

`draw_square(x - size, y, size)`
วาดสี่เหลี่ยมด้านซ้าย

`draw_square(x + size, y, size)`
วาดสี่เหลี่ยมด้านขวา

ผลการทำงาน

โปรแกรมจะแสดงเครื่องหมายบวกจำนวน 5 รูป แต่ละรูปมีความเร็วและทิศทางแตกต่างกัน เมื่อเครื่องหมายบวกเคลื่อนที่ถึงขอบหน้าจอจะเปลี่ยนทิศทาง และเมื่อเลื่อนเมาส์เข้าใกล้ เครื่องหมายบวกจะมีขนาดใหญ่ขึ้น

Error ที่พบและวิธีแก้ไข

1. ลืมกำหนด `number_of_positive`
จะทำให้เกิดปัญหาเนื่องจากโปรแกรมไม่รู้จำนวนรอบที่ต้องวน

วิธีแก้ไข:

number_of_positive = 5

2. ลืมเพิ่มค่า `count`
หากไม่มีคำสั่ง

count = count + 1

จะเกิด Infinite Loop เพราะ `count` มีค่าเดิมตลอดเวลา

3. List มีข้อมูลน้อยกว่าจำนวนที่กำหนด
หาก `number_of_positive` มีค่า 5 แต่ List มีข้อมูลเพียง 3 ค่า อาจเกิด Index Error

วิธีแก้ไขคือจำนวนข้อมูลใน List ควรสอดคล้องกับจำนวนเครื่องหมายบวก

---

3. Text Mode ข้อ a: หา Max, Min และ Average

วัตถุประสงค์ของข้อ

ข้อนี้เขียนฟังก์ชันเพื่อรับ List ของตัวเลขและหาค่าสูงสุด ค่าต่ำสุด และค่าเฉลี่ย โดยใช้ `while` แทน `for loop`

โค้ด

```
def find_max(arr):
    max_value = arr[0]
    i = 1

    while i < len(arr):
        if arr[i] > max_value:
            max_value = arr[i]

        i = i + 1
```

```
    return max_value


def find_min(arr):
    min_value = arr[0]
    i = 1

    while i < len(arr):
        if arr[i] < min_value:
            min_value = arr[i]

        i = i + 1

    return min_value


def find_average(arr):
    total = 0
    i = 0

    while i < len(arr):
        total = total + arr[i]
        i = i + 1

    return total / len(arr)


numbers = [10, 25, 7, 40, 15]

print("Max =", find_max(numbers))
print("Min =", find_min(numbers))
print("Average =", find_average(numbers))
```

อธิบายการทำงาน

ส่วนหา Max

`def find_max(arr):`
สร้างฟังก์ชันรับ List ชื่อ `arr`

`max_value = arr[0]`
กำหนดสมาชิกตัวแรกเป็นค่าสูงสุดเริ่มต้น

`i = 1`
เริ่มตรวจสอบตั้งแต่สมาชิกตัวที่ 2 เพราะสมาชิกตัวแรกถูกนำไปเป็นค่าเริ่มต้นแล้ว

`while i < len(arr):`
วนตรวจสอบสมาชิกทุกตัวใน List

`if arr[i] > max_value:`
ตรวจสอบว่าค่าปัจจุบันมากกว่าค่าสูงสุดเดิมหรือไม่

`max_value = arr[i]`
หากมากกว่า ให้นำค่าปัจจุบันมาเป็นค่าสูงสุดใหม่

`i = i + 1`
เลื่อนไปตรวจสอบสมาชิกตัวถัดไป

`return max_value`
ส่งค่ามากที่สุดกลับจากฟังก์ชัน

ส่วนหา Min

หลักการเหมือนกับการหา Max แต่เปลี่ยนเงื่อนไขเป็น

if arr[i] < min_value:

หากค่าปัจจุบันน้อยกว่า ก็จะเปลี่ยนค่า Min

ส่วนหา Average

`total = 0`
สร้างตัวแปรเก็บผลรวม

`while i < len(arr):`
วนผ่านสมาชิกทุกตัว

`total = total + arr[i]`
นำค่าของสมาชิกแต่ละตัวมาบวกสะสม

`return total / len(arr)`
นำผลรวมหารด้วยจำนวนสมาชิกทั้งหมดเพื่อหาค่าเฉลี่ย

Error ที่พบและวิธีแก้ไข

Empty List
หากส่ง List ว่างเข้าไป เช่น `[]` จะเกิดปัญหาที่ `arr[0]`

ดังนั้นข้อมูลที่ส่งเข้าฟังก์ชันควรมีอย่างน้อย 1 ค่า

---

4. Text Mode ข้อ b: หา Index ของ Max และ Min

วัตถุประสงค์ของข้อ

ข้อนี้ไม่ได้ต้องการเพียงค่ามากที่สุดหรือน้อยที่สุด แต่ต้องการทราบตำแหน่งหรือ Index ของค่านั้นใน List

โค้ด

```
def find_max_index(arr):
    max_index = 0
    i = 1

    while i < len(arr):
        if arr[i] > arr[max_index]:
            max_index = i

        i = i + 1

    return max_index


def find_min_index(arr):
    min_index = 0
    i = 1

    while i < len(arr):
        if arr[i] < arr[min_index]:
            min_index = i

        i = i + 1

    return min_index


numbers = [10, 25, 7, 40, 15]

print("Max index =", find_max_index(numbers))
print("Min index =", find_min_index(numbers))
```

อธิบายการทำงาน

`max_index = 0`
กำหนดให้ตำแหน่งแรกเป็นตำแหน่งของค่าสูงสุดในตอนเริ่มต้น

`if arr[i] > arr[max_index]:`
เปรียบเทียบค่าปัจจุบันกับค่าที่อยู่ในตำแหน่งสูงสุดเดิม

`max_index = i`
หากค่าปัจจุบันมากกว่า ให้เก็บตำแหน่งปัจจุบันเป็น Index ใหม่

การหา Min ใช้หลักการเดียวกัน แต่เปลี่ยนจาก `>` เป็น `<`

สำหรับ

numbers = [10, 25, 7, 40, 15]

ตำแหน่งคือ

Index: 0 1 2 3 4
Value: 10 25 7 40 15

ดังนั้น Max คือ 40 อยู่ที่ Index 3 และ Min คือ 7 อยู่ที่ Index 2

Error ที่พบและวิธีแก้ไข

หากเขียน

arr[max_index]

โดยที่ `max_index` มีค่ามากกว่าขนาดของ List จะเกิด Index Error ดังนั้นต้องควบคุม `i` ด้วย

while i < len(arr):

---

5. Text Mode ข้อ c: ค้นหาค่าใน Array และ Return Index

วัตถุประสงค์ของข้อ

ข้อนี้สร้างฟังก์ชันสำหรับค้นหาค่าที่ต้องการใน List หากพบให้ส่งตำแหน่ง Index กลับมา แต่หากไม่พบให้ส่งค่า `-1`

โค้ด

```
def search_value(arr, target):
    i = 0

    while i < len(arr):
        if arr[i] == target:
            return i

        i = i + 1

    return -1


numbers = [10, 25, 7, 40, 15]

target = int(input("Enter value: "))

index = search_value(numbers, target)

if index == -1:
    print("Not found")
else:
    print("Found at index =", index)
```

อธิบายการทำงาน

`def search_value(arr, target):`
สร้างฟังก์ชันรับ List และค่าที่ต้องการค้นหา

`i = 0`
เริ่มต้นตรวจสอบจาก Index 0

`while i < len(arr):`
วนตรวจสอบสมาชิกทุกตัว

`if arr[i] == target:`
ตรวจสอบว่าสมาชิกปัจจุบันมีค่าเท่ากับค่าที่ต้องการหรือไม่

`return i`
หากพบ ให้ส่งตำแหน่ง Index กลับทันที

`return -1`
หากตรวจสอบครบทุกตัวแล้วไม่พบ ให้ส่งค่า -1 เพื่อแทนว่าไม่พบข้อมูล

`target = int(input("Enter value: "))`
รับค่าจากผู้ใช้และแปลงเป็นจำนวนเต็ม

Error ที่พบและวิธีแก้ไข

ValueError
หากผู้ใช้กรอกข้อความแทนตัวเลข อาจเกิด Error จาก

int(input())

วิธีแก้ไขคือผู้ใช้ต้องกรอกจำนวนเต็ม หรือสามารถเพิ่มการตรวจสอบข้อมูลในโปรแกรมได้

---

6. Text Mode ข้อ d: นับจำนวนสมาชิกที่มีค่ามากกว่า Average

วัตถุประสงค์ของข้อ

ข้อนี้หาค่าเฉลี่ยของข้อมูลก่อน จากนั้นตรวจสอบสมาชิกทุกตัวและนับว่ามีกี่ตัวที่มีค่ามากกว่าค่าเฉลี่ย

โค้ด

```
def find_average(arr):
    total = 0
    i = 0

    while i < len(arr):
        total = total + arr[i]
        i = i + 1

    return total / len(arr)


def count_higher_than_average(arr):
    average = find_average(arr)

    count = 0
    i = 0

    while i < len(arr):
        if arr[i] > average:
            count = count + 1

        i = i + 1

    return count
```

numbers = [10, 25, 7, 40, 15]

print(count_higher_than_average(numbers))

อธิบายการทำงาน

`average = find_average(arr)`
เรียกฟังก์ชันเพื่อหาค่าเฉลี่ยของ List

`count = 0`
สร้างตัวแปรสำหรับนับจำนวนสมาชิก

`if arr[i] > average:`
ตรวจสอบว่าสมาชิกปัจจุบันมีค่ามากกว่าค่าเฉลี่ยหรือไม่

`count = count + 1`
หากมากกว่า ให้เพิ่มตัวนับ 1

`return count`
ส่งจำนวนสมาชิกที่มากกว่าค่าเฉลี่ยกลับมา

สำหรับ

[10, 25, 7, 40, 15]

ค่าเฉลี่ยคือ 19.4

สมาชิกที่มากกว่าค่าเฉลี่ยคือ 25 และ 40 ดังนั้นผลลัพธ์คือ 2

Error ที่พบและวิธีแก้ไข

หาก List ว่าง จะไม่สามารถหารด้วย `len(arr)` ได้ เนื่องจากจะเกิดการหารด้วยศูนย์ ดังนั้นควรตรวจสอบว่า List มีข้อมูลก่อนใช้งาน

---

7. Text Mode ข้อ e: ตรวจสอบว่า Array เรียงจากน้อยไปมากหรือไม่

วัตถุประสงค์ของข้อ

ข้อนี้ตรวจสอบว่าแต่ละสมาชิกใน List มีค่าเรียงจากน้อยไปมากหรือไม่ โดยเปรียบเทียบสมาชิกที่อยู่ติดกันทีละคู่

โค้ด

```
def is_increasing(arr):
    i = 0

    while i < len(arr) - 1:

        if arr[i] > arr[i + 1]:
            return False

        i = i + 1

    return True


numbers = [10, 20, 30, 40, 50]

if is_increasing(numbers):
    print("Array is increasing")
else:
    print("Array is not increasing")
```

อธิบายการทำงาน

`i = 0`
เริ่มเปรียบเทียบจากสมาชิกตัวแรก

`while i < len(arr) - 1:`
วนจนถึงสมาชิกตัวก่อนสุดท้าย เนื่องจากต้องเปรียบเทียบ `arr[i]` กับ `arr[i + 1]`

`if arr[i] > arr[i + 1]:`
หากสมาชิกตัวซ้ายมีค่ามากกว่าสมาชิกตัวขวา แสดงว่าข้อมูลไม่ได้เรียงจากน้อยไปมาก

`return False`
หยุดการทำงานและส่งค่า False กลับทันที

`i = i + 1`
เลื่อนไปเปรียบเทียบสมาชิกคู่ถัดไป

`return True`
หากตรวจสอบครบทุกคู่และไม่พบค่าที่เรียงผิด แสดงว่า List เรียงจากน้อยไปมาก

Error ที่พบและวิธีแก้ไข

หากใช้เงื่อนไข

while i < len(arr):

อาจเกิด Index Error ในคำสั่ง

arr[i + 1]

เมื่อ `i` เป็น Index สุดท้าย เพราะไม่มีสมาชิกตัวถัดไป ดังนั้นจึงต้องใช้

while i < len(arr) - 1:

---

8. Text Mode: การหาค่าความแปรปรวน (Variance)

วัตถุประสงค์ของข้อ

ข้อนี้เขียนฟังก์ชันที่รับพารามิเตอร์เป็น Array หรือ List ของจำนวนเต็ม และคำนวณค่าความแปรปรวน หรือ Variance ด้วยตนเอง โดยไม่ใช้ Built-in Function สำหรับคำนวณค่าเฉลี่ยหรือ Variance

โค้ด

```
def find_variance(arr):

    total = 0
    i = 0

    while i < len(arr):
        total = total + arr[i]
        i = i + 1

    average = total / len(arr)

    total_square = 0
    i = 0

    while i < len(arr):
        difference = arr[i] - average
        total_square = total_square + difference * difference
        i = i + 1
```

```
    variance = total_square / len(arr)

    return variance


numbers = [10, 20, 30, 40, 50]

result = find_variance(numbers)

print("Variance =", result)
```

อธิบายการทำงาน

`def find_variance(arr):`
สร้างฟังก์ชันที่รับ List ของตัวเลขเป็นพารามิเตอร์

`total = 0`
สร้างตัวแปรเก็บผลรวมของข้อมูล

`i = 0`
กำหนด Index เริ่มต้น

`while i < len(arr):`
วนผ่านสมาชิกทุกตัว

`total = total + arr[i]`
นำสมาชิกแต่ละตัวมาบวกสะสม

`i = i + 1`
เลื่อนไปยังสมาชิกตัวถัดไป

`average = total / len(arr)`
นำผลรวมหารด้วยจำนวนสมาชิกเพื่อหาค่าเฉลี่ย

`total_square = 0`
สร้างตัวแปรสำหรับเก็บผลรวมของค่าความแตกต่างยกกำลังสอง

`difference = arr[i] - average`
หาความแตกต่างระหว่างข้อมูลแต่ละตัวกับค่าเฉลี่ย

`difference * difference`
คำนวณค่าความแตกต่างยกกำลังสอง โดยไม่ใช้ฟังก์ชัน Built-in

`total_square = total_square + difference * difference`
นำค่าที่คำนวณได้มาบวกสะสม

`variance = total_square / len(arr)`
นำผลรวมของค่าความแตกต่างยกกำลังสองหารด้วยจำนวนข้อมูล

`return variance`
ส่งค่าความแปรปรวนกลับจากฟังก์ชัน

ตัวอย่างการคำนวณ

ข้อมูล:

[10, 20, 30, 40, 50]

ค่าเฉลี่ย:

(10 + 20 + 30 + 40 + 50) / 5 = 30

ความแตกต่างยกกำลังสอง:

(10 - 30)² = 400
(20 - 30)² = 100
(30 - 30)² = 0
(40 - 30)² = 100
(50 - 30)² = 400

ผลรวม:

400 + 100 + 0 + 100 + 400 = 1000

Variance:

1000 / 5 = 200

ดังนั้นผลลัพธ์คือ

Variance = 200.0

Error ที่พบและวิธีแก้ไข

1. ลืมกำหนดค่า `i = 0` ใหม่ก่อน while รอบที่สอง
หลังจาก While รอบแรก `i` จะมีค่าเท่ากับจำนวนสมาชิก หากไม่กำหนด

i = 0

ใหม่ While รอบที่สองจะไม่ทำงาน

2. List ว่าง
หาก List ไม่มีข้อมูล จะเกิดปัญหาในการหารด้วย `len(arr)` ซึ่งเท่ากับ 0

ดังนั้นควรส่งข้อมูลที่มีสมาชิกอย่างน้อย 1 ค่าเข้าไปในฟังก์ชัน

สรุปผลการปฏิบัติการ

จากการทำ Lab 5 ได้เรียนรู้การใช้ List หรือ Array ในภาษา Python การเข้าถึงข้อมูลด้วย Index และการใช้ `while loop` เพื่อวนทำงานกับสมาชิกแต่ละตัว นอกจากนี้ยังได้ฝึกสร้างฟังก์ชันเพื่อประมวลผลข้อมูล ได้แก่ การหาค่าสูงสุด ค่าต่ำสุด ค่าเฉลี่ย การหา Index การค้นหาข้อมูล การนับข้อมูลตามเงื่อนไข การตรวจสอบการเรียงลำดับ และการคำนวณ Variance ด้วยตนเอง

ในส่วนของ Graphic Mode ได้เรียนรู้การใช้ตัวแปรและ List เพื่อควบคุมตำแหน่งของวัตถุ รวมถึงการสร้าง Animation และ Interaction ระหว่างเมาส์กับวัตถุ โดยทุกส่วนที่ต้องวนซ้ำใช้ `while loop` ตามเงื่อนไขของการปฏิบัติการ